



UNIVERSIDADE CATÓLICA DO SALVADOR

CHECKLIST DE TESTE ESTÁTICO DE CÓDIGO (REVISÃO MANUAL)

A equipe deve preencher e entregar um checklist de verificação estática do código-fonte respondendo e documentando as seguintes questões:

Todas as variáveis estão inicializadas antes do uso?

MAIN

Sim, todas as variáveis foram iniciadas antes do uso, menos o “chosen” pois foi instanciada como null para uso posterior.

TODAS AS CLASSE:

Todas as variáveis foram inicializadas.

Há variáveis declaradas e nunca usadas?

TODAS AS CLASSE:

Todas as variáveis declaradas foram usadas.

Existe código inacessível?

TODAS AS CLASSE (exceto Item):

Não possui código inacessível

ITEM

Sim, os blocos referentes aos itens x-atk, x-def e x-spd estão dentro da condição que verifica se o item é uma potion, quando o item é um desses três, esses blocos não são executados.

Há código duplicado?

MAIN

A criação de enemy e enemy2 é praticamente igual.

TODAS AS CLASSE (exceto main):

Não possui código duplicado

Existem erros de sintaxe?

TODAS AS CLASSE:

Não possui erro sintático encontrado.

Há erros de lógica que quebram regras do negócio?

MAIN

Sim, o while (chosen == null) ao ser escolhido um número inválido continua o game ao invés de pedir ao usuário escolher outro número.

BATTLE

No método atacar(), são utilizados == para comparar Strings, o que pode fazer a comparação não funcionar como esperado e a condição rollEfeitoStatus < 25 && rollEfeitoStatus < 30 é redundante. Além disso no método curarFimDeTurno(), o HP só é atualizado quando cura <= 0, quando a cura possui um valor normal positivo, o setHp() não é executado.

CAMPO

Sim. Existe uma diferença na escrita entre "canteiro central" e "canteiro Central", pois o equals() diferencia maiúsculas e minúsculas.

POKESAL

Sim. O método receberDano() verifica se o HP é menor que zero antes de aplicar o dano:

```
if (this.hp < 0) {  
    this.hp = 0;  
} else {  
    this.hp -= danoFinal;  
}
```

O HP pode ficar negativo após a aplicação do dano.

Há erros de tipagem?

MAIN

Não foi identificado erro de tipagem

BATTLE

Há um uso de "==" ao invés de ".equals()" em um String.

O fluxo de controle é válido (sem loops infinitos, condições impossíveis)?

MAIN

Não totalmente. O while (chosen == null) possui um problema porque a opção é lida apenas uma vez. Caso seja inválida, o usuário não consegue informar uma nova opção dentro desse fluxo.

ITEM

O fluxo apresenta problema porque os `else if` dos itens `x-atk`, `x-def` e `x-spд` estão subordinados à condição da `potion`.

POKESAL

O fluxo de controle é válido e não possui loops infinitos. Porém, a condição utilizada em `receberDano()` está posicionada de forma inadequada, permitindo que o HP fique negativo após a aplicação do dano.

Outros tipos de erros identificados?

CAMPO

A comparação do cenário depende exatamente da escrita da `String`. Também pode existir uma inconsistência na regra de cura, pois o cálculo utiliza o HP atual para calcular 5%, `int cura = (int) (p.getHp() * 0.05)`, deveria utilizar `getMaxHp()`.

POKESAL

Os métodos `setHp()`, `setMaxHp()`, `setAtk()`, `setDef()` e `setSpд()` não possuem validações. Assim, podem receber valores negativos ou valores inconsistentes, como um HP maior que o `maxHp`.

TRAINER

O método `getBag()` retorna diretamente a lista interna da mochila. Isso permite que outras classes adicionem ou removam itens diretamente, podendo contornar regras implementadas na própria classe, como o limite de 2 itens.